

Práctica guiada: ViewModels en ASP.NET Core MVC

Programación Web III – ASP.NET Core MVC · Proyecto EntradasMvc

Estudiante: Alejandro Rojas Espinoza

Materia: Programación Web III

Fecha: 01/10/2026

Repositorio: <https://github.com/alejandrorojas03010-star/EntradasMvc>

Estructura final del proyecto

```
EntradasMvc
├── Controllers
│   └── EntradasController.cs
├── Models
│   └── Cotizacion.cs
├── ViewModels
│   ├── CotizacionInputViewModel.cs
│   └── ResultadoCotizacionViewModel.cs
├── Views
│   ├── Entradas
│   │   ├── Index.cshtml
│   │   └── Resultado.cshtml
└── Program.cs
```

Flujo

```
Index.cshtml —POST—> CotizacionInputViewModel (Model Binding + Data Annotations)
                        |
                        └── EntradasController.Calcular
                                |
                                └── Cotizacion (reglas de negocio: subtotal, 10 % de descuento, total)
                                        |
                                        └── ResultadoCotizacionViewModel (cotización + evento, fecha, mensaje)
                                                |
                                                └── Resultado.cshtml
```

Ejercicio individual (punto 21)

- `CotizacionInputViewModel` tiene la propiedad `TipoEntrada` con `[Required]` y `[Display(Name = "Tipo de entrada")]`.
- `Index.cshtml` muestra un `<select asp-for="TipoEntrada" asp-items="...">` con las opciones **General** y **VIP**.
- El valor seleccionado llega a `Calcular` por Model Binding, se copia a `Cotizacion.TipoEntrada` y la vista de resultado muestra `Tipo de entrada: VIP`.
- No se modificó ninguna base de datos ni se creó ninguna migración.

Preguntas para entregar (punto 22)

1. ¿Cuál es la diferencia principal entre Model y ViewModel?

El **Model** representa los datos y el comportamiento del dominio: contiene las reglas de negocio (en esta práctica, `Cotizacion` calcula el subtotal, el descuento y el total). El **ViewModel** es una clase pensada para una vista concreta: contiene solo los datos que esa pantalla necesita mostrar o enviar, aunque vengan de varias fuentes (por ejemplo, `ResultadoCotizacionViewModel` junta la cotización con el evento, la fecha y el mensaje).

2. ¿Cómo sabe una vista qué tipo de ViewModel recibe?

Por la directiva `@model` al inicio de la vista. Por ejemplo, `@model EntradasMvc.ViewModels.ResultadoCotizacionViewModel` hace que la vista sea fuertemente tipada: `Model` es de ese tipo, Visual Studio ofrece IntelliSense con sus propiedades y, si se usa una propiedad que no existe (como `@Model.Nombre`), el proyecto no compila.

3. ¿Qué función cumple Model Binding?

Toma los datos de la solicitud HTTP (campos del formulario, ruta, query string) y crea automáticamente el objeto que recibe la acción, asignando cada valor a la propiedad del mismo nombre y convirtiendo los tipos. Por ejemplo, `Cliente=Ana Pérez&Cantidad=5` se convierte en un `CotizacionInputViewModel` con `Cliente = "Ana Pérez"` y `Cantidad = 5`. También registra en `ModelState` los errores de conversión y de validación.

4. ¿Puede un ViewModel recibir datos desde el navegador? Explique.

Sí. Un ViewModel no solo sirve para enviar datos del controlador a la vista; también sirve en sentido contrario. En esta práctica, el formulario de `Index.cshtml` envía sus datos por POST y la acción `Calcular(CotizacionInputViewModel viewModel)` los recibe ya cargados en ese ViewModel gracias a Model Binding, y además validados con sus Data Annotations.

5. ¿Qué son las Data Annotations?

Son atributos de .NET (`System.ComponentModel.DataAnnotations`) que se colocan sobre las propiedades para describirlas y validarlas. Por ejemplo, `[Required]` hace que el dato sea obligatorio, `[StringLength]` limita la longitud del texto, `[Range]` restringe un valor a un intervalo y `[Display]` define el nombre que se muestra en la interfaz. ASP.NET Core las usa para validar en el servidor (`ModelState.IsValid`) y, con los Tag Helpers, también para generar la validación del lado del cliente.

6. ¿Por qué `Evento` puede pertenecer al ViewModel de resultado y no necesariamente a `Cotizacion` ?

Porque `Evento` (al igual que `FechaEvento` y `Mensaje`) es un dato que solo necesita la pantalla de resultado para presentar la información. No interviene en el cálculo del subtotal, del descuento ni del total, así que no es una regla ni un dato del dominio de la cotización. Ponerlo en el ViewModel mantiene a `Cotizacion` enfocada en el negocio.

7. ¿Dónde debe permanecer la regla del descuento y por qué?

En el Model `Cotizacion ( Descuento => Cantidad >= 5 ? Subtotal * 0.10m : 0m )`, porque es una regla de negocio. Si estuviera en un ViewModel o en una vista, quedaría atada a una sola pantalla, habría que repetirla en otras y sería más fácil que quedara inconsistente. En el Model hay una única fuente de verdad para cualquier vista o controlador que use la cotización.

8. ¿Un ViewModel representa obligatoriamente una tabla de la base de datos?

No. Un ViewModel se diseña según lo que necesita una vista, no según la base de datos. No necesita estar en el `DbContext` ni tener una migración, puede combinar datos de varias fuentes y no reemplaza al Model de dominio.

9. Explique el recorrido completo desde que se pulsa Calcular hasta que aparece el resultado.

1. El usuario completa el formulario de `Index.cshtml` (cliente, cantidad y tipo de entrada) y pulsa **Calcular**.
2. El navegador envía un POST a `/Entradas/Calcular` con los campos del formulario y el token antifalsificación.
3. `[ValidateAntiForgeryToken]` verifica el token.
4. Model Binding crea un `CotizacionInputViewModel` con los valores recibidos y las Data Annotations lo validan.
5. Si `ModelState.IsValid` es `false`, el controlador devuelve `View("Index", viewModel)` y el formulario se vuelve a mostrar con los datos ingresados y los mensajes de error.
6. Si es válido, el controlador crea un `Cotizacion` copiando los datos del ViewModel de entrada. El Model calcula `Subtotal`, `Descuento` y `Total`.
7. El controlador crea un `ResultadoCotizacionViewModel` con la cotización y los datos de presentación (`Evento`, `FechaEvento`, `Mensaje`).
8. `return View("Resultado", resultadoViewModel)` renderiza `Resultado.cshtml`, que es fuertemente tipada a `ResultadoCotizacionViewModel` y muestra los datos (por ejemplo, Bs 250.00 de subtotal, Bs 25.00 de descuento y Bs 225.00 de total para 5 entradas).

10. ¿Qué ventaja tiene utilizar un ViewModel de entrada en un formulario?

Define exactamente qué datos acepta esa pantalla (aquí solo `Cliente`, `Cantidad` y `TipoEntrada`) y concentra en un solo lugar las validaciones propias del formulario. Así, el usuario no puede enviar propiedades que no corresponden (lo que evita el *overposting*), el Model de dominio no se llena de reglas de la interfaz y la vista queda más simple, segura y fácil de mantener.